In [2]:
```
# ==========================================================
# 4. COMBINED MODEL NOTEBOOK (MULTIMODAL CLASSIFICATION)
# ==========================================================

# ------------------------------------------------
# 4.1 IMPORTS AND SETUP
# ------------------------------------------------
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
import os
import pickle
import librosa
from sklearn.model_selection import train_test_split, cross_val_score, StratifiedKF
from sklearn.ensemble import RandomForestClassifier, VotingClassifier
from sklearn.metrics import classification_report, confusion_matrix, accuracy_score
from sklearn.preprocessing import StandardScaler
from sklearn.pipeline import Pipeline
from scipy.sparse import hstack, csr_matrix
import warnings
warnings.filterwarnings('ignore')

print("Libraries imported successfully")

# ------------------------------------------------
# 4.2 LOAD DATA AND TRAINED MODELS
# ------------------------------------------------
print("\nLoading dataset and trained models...")
data_path = "data/overview-of-recordings.csv"
data = pd.read_csv(data_path)
print(f"Dataset shape: {data.shape}")
print(f"Number of unique prompts: {data['prompt'].nunique()}")

# Ensure models directory exists
os.makedirs('models', exist_ok=True)

# Load the trained models
try:
    text_model_path = 'models/text_classification_model.pkl'
    audio_model_path = 'models/audio_classification_model.pkl'

    if os.path.exists(text_model_path) and os.path.exists(audio_model_path):
        with open(text_model_path, 'rb') as f:
            text_model = pickle.load(f)
        print("Loaded text classification model")

        with open(audio_model_path, 'rb') as f:
            audio_model = pickle.load(f)
        print("Loaded audio classification model")

        models_loaded = True
    else:
        missing = []
```

```
        if not os.path.exists(text_model_path):
            missing.append("text_classification_model.pkl")
        if not os.path.exists(audio_model_path):
            missing.append("audio_classification_model.pkl")

        print(f"Could not find models: {', '.join(missing)}")
        print("Please run the text and audio notebooks first to generate these mode
        models_loaded = False
except Exception as e:
    print(f"Error loading models: {e}")
    models_loaded = False

# ------------------------------------------------
# 4.3 FEATURE EXTRACTION AND PREPARATION
# ------------------------------------------------
if models_loaded:
    print("\nPreparing features for combined model...")

    # Process text data for text model predictions
    text_data = data['phrase'].fillna("")

    # Process audio data for audio model predictions
    # Define audio feature extraction function (same as in the audio notebook)
    def extract_features(file_path):
        try:
            # Load audio file
            y, sr = librosa.load(file_path, sr=None)

            # Extract features
            # Time-domain features
            duration = librosa.get_duration(y=y, sr=sr)
            zero_crossing_rate = np.mean(librosa.feature.zero_crossing_rate(y))
            zero_crossing_rate_std = np.std(librosa.feature.zero_crossing_rate(y))
            rms = np.mean(librosa.feature.rms(y=y))
            rms_std = np.std(librosa.feature.rms(y=y))

            # Frequency-domain features
            spectral_centroid = np.mean(librosa.feature.spectral_centroid(y=y, sr=s
            spectral_centroid_std = np.std(librosa.feature.spectral_centroid(y=y, s
            spectral_bandwidth = np.mean(librosa.feature.spectral_bandwidth(y=y, sr
            spectral_bandwidth_std = np.std(librosa.feature.spectral_bandwidth(y=y,
            spectral_rolloff = np.mean(librosa.feature.spectral_rolloff(y=y, sr=sr)
            spectral_rolloff_std = np.std(librosa.feature.spectral_rolloff(y=y, sr=

            # Mel-frequency cepstral coefficients
            mfccs = librosa.feature.mfcc(y=y, sr=sr, n_mfcc=13)
            mfccs_mean = np.mean(mfccs, axis=1)
            mfccs_std = np.std(mfccs, axis=1)

            # Tempo
            tempo, _ = librosa.beat.beat_track(y=y, sr=sr)

            # Chroma features
            chroma = librosa.feature.chroma_stft(y=y, sr=sr)
            chroma_mean = np.mean(chroma, axis=1)
            chroma_std = np.std(chroma, axis=1)
```

```
        # Combine all features into a flat array
        feature_list = [
            duration, zero_crossing_rate, zero_crossing_rate_std,
            rms, rms_std, spectral_centroid, spectral_centroid_std,
            spectral_bandwidth, spectral_bandwidth_std,
            spectral_rolloff, spectral_rolloff_std,
            tempo
        ]

        # Add MFCCs
        feature_list.extend(mfccs_mean)
        feature_list.extend(mfccs_std)

        # Add chroma features
        feature_list.extend(chroma_mean)
        feature_list.extend(chroma_std)

        return feature_list

    except Exception as e:
        print(f"Error extracting features from {file_path}: {e}")
        return None

# Define base audio directory
audio_base_dir = "data/recordings/"

# Function to find audio file in the correct subdirectory
def find_audio_file(file_name):
    # Check each subdirectory
    for subdir in ['train', 'test', 'validate']:
        file_path = os.path.join(audio_base_dir, subdir, file_name)
        if os.path.exists(file_path):
            return file_path
    return None

# Extract audio features
print("\nExtracting audio features for combined model...")

audio_features = []
valid_indices = []

# Process files to extract features
total_files = len(data)
for i, row in data.iterrows():
    file_path = find_audio_file(row['file_name'])

    # Check if file exists
    if file_path:
        features = extract_features(file_path)

        if features:
            audio_features.append(features)
            valid_indices.append(i)

    # Print progress
```

```
        if (i+1) % 50 == 0 or (i+1) == total_files:
            print(f"Processed {i+1}/{total_files} files")

    # Convert to numpy array
    if len(audio_features) > 0:
        X_audio = np.array(audio_features)

        # Filter text data to match valid audio indices
        text_data_filtered = text_data.iloc[valid_indices].reset_index(drop=True)
        y_filtered = data['prompt'].iloc[valid_indices].reset_index(drop=True)

        print(f"\nProcessed {len(audio_features)} valid audio samples")
        features_ready = True
    else:
        print("\nNo valid audio features extracted. Creating dummy data for demonst

        # Create dummy audio features for demonstration
        sample_count = min(100, len(data))
        feature_count = 61  # Match the number of features in extract_features

        # Generate random features
        np.random.seed(42)
        X_audio = np.random.randn(sample_count, feature_count)

        # Sample indices for text data
        valid_indices = np.random.choice(len(data), sample_count, replace=False)
        valid_indices.sort()  # Sort to maintain order

        # Filter text data
        text_data_filtered = text_data.iloc[valid_indices].reset_index(drop=True)
        y_filtered = data['prompt'].iloc[valid_indices].reset_index(drop=True)

        print(f"Created {sample_count} dummy audio samples for demonstration")
        features_ready = True
else:
    features_ready = False

# ------------------------------------------------
# 4.4 MODEL COMBINATION AND EVALUATION
# ------------------------------------------------
if models_loaded and features_ready:
    print("\n--- MODEL COMBINATION AND EVALUATION ---")

    # Get predictions from individual models
    print("\nGenerating predictions from individual models...")

    # Text model predictions
    if hasattr(text_model, 'predict_proba'):
        text_probs = text_model.predict_proba(text_data_filtered)
        print(f"Text model probability shape: {text_probs.shape}")
    else:
        print("Text model does not support probability predictions, using decision
        text_probs = text_model.decision_function(text_data_filtered)
        # Convert decision function to pseudo-probabilities
        if text_probs.ndim == 1:
            # Binary classification
```

```
            text_probs = np.column_stack([1 - text_probs, text_probs])
        else:
            # Multi-class
            text_probs = np.exp(text_probs) / np.sum(np.exp(text_probs), axis=1, ke
        print(f"Text model transformed probabilities shape: {text_probs.shape}")

    # Audio model predictions
    if hasattr(audio_model, 'predict_proba'):
        audio_probs = audio_model.predict_proba(X_audio)
        print(f"Audio model probability shape: {audio_probs.shape}")
    else:
        print("Audio model does not support probability predictions, using decision
        audio_probs = audio_model.decision_function(X_audio)
        # Convert decision function to pseudo-probabilities
        if audio_probs.ndim == 1:
            # Binary classification
            audio_probs = np.column_stack([1 - audio_probs, audio_probs])
        else:
            # Multi-class
            audio_probs = np.exp(audio_probs) / np.sum(np.exp(audio_probs), axis=1,
        print(f"Audio model transformed probabilities shape: {audio_probs.shape}")

    # Combine predictions
    print("\nCombining predictions...")

    # Method 1: Simple averaging of probabilities
    print("\nMethod 1: Simple probability averaging")
    combined_probs = (text_probs + audio_probs) / 2
    combined_predictions = np.argmax(combined_probs, axis=1)

    # Convert numeric predictions to class labels
    classes = text_model.classes_ if hasattr(text_model, 'classes_') else audio_mod
    combined_predictions = classes[combined_predictions]

    # Evaluate combined model
    print("\nCombined Model Results:")
    combined_accuracy = accuracy_score(y_filtered, combined_predictions)
    print(f"Accuracy: {combined_accuracy:.4f}")
    print(classification_report(y_filtered, combined_predictions))

    # Method 2: Train a meta-classifier
    print("\nMethod 2: Meta-classifier approach")

    # Create a combined feature set (concatenation of probabilities)
    combined_features = np.concatenate([text_probs, audio_probs], axis=1)

    # Split data for meta-classifier
    X_train, X_test, y_train, y_test = train_test_split(
        combined_features, y_filtered,
        test_size=0.2, random_state=42, stratify=y_filtered
    )

    # Train a meta-classifier
    meta_classifier = RandomForestClassifier(n_estimators=100, random_state=42)
    meta_classifier.fit(X_train, y_train)
```

```
# Evaluate
meta_predictions = meta_classifier.predict(X_test)
meta_accuracy = accuracy_score(y_test, meta_predictions)

print("\nMeta-Classifier Results:")
print(f"Accuracy: {meta_accuracy:.4f}")
print(classification_report(y_test, meta_predictions))

# Plot confusion matrix for meta-classifier
plt.figure(figsize=(12, 10))
cm = confusion_matrix(y_test, meta_predictions)
sns.heatmap(cm, annot=True, fmt='d', cmap='Blues',
            xticklabels=classes,
            yticklabels=classes)
plt.xlabel('Predicted')
plt.ylabel('True')
plt.title('Confusion Matrix: Meta-Classifier')
plt.xticks(rotation=45)
plt.tight_layout()
plt.show()

# ------------------------------------------------
# 4.5 COMPARE INDIVIDUAL VS COMBINED MODELS
# ------------------------------------------------
print("\n--- COMPARING INDIVIDUAL VS COMBINED MODELS ---")

# Get individual model predictions on the test set
text_predictions = text_model.predict(text_data_filtered.iloc[len(X_train):].re
audio_predictions = audio_model.predict(X_audio[len(X_train):])

# Calculate accuracies
text_accuracy = accuracy_score(y_test, text_predictions)
audio_accuracy = accuracy_score(y_test, audio_predictions)

# Compare accuracies
print("\nAccuracy Comparison:")
print(f"Text-only model: {text_accuracy:.4f}")
print(f"Audio-only model: {audio_accuracy:.4f}")
print(f"Simple averaging: {combined_accuracy:.4f}")
print(f"Meta-classifier: {meta_accuracy:.4f}")

# Plot comparison
plt.figure(figsize=(10, 6))
models = ['Text-only', 'Audio-only', 'Simple averaging', 'Meta-classifier']
accuracies = [text_accuracy, audio_accuracy, combined_accuracy, meta_accuracy]

plt.bar(models, accuracies)
plt.ylim(0, 1.0)
plt.ylabel('Accuracy')
plt.title('Model Accuracy Comparison')

for i, v in enumerate(accuracies):
    plt.text(i, v + 0.02, f"{v:.4f}", ha='center')

plt.tight_layout()
plt.show()
```

```
# ----------------------------------------------
# 4.6 SAVE THE COMBINED MODEL
# ----------------------------------------------
print("\nSaving the meta-classifier model...")

# Since the meta-classifier needs both base models, save all three
combined_model = {
    'text_model': text_model,
    'audio_model': audio_model,
    'meta_classifier': meta_classifier
}

with open('models/combined_classification_model.pkl', 'wb') as f:
    pickle.dump(combined_model, f)

print("Combined model saved as 'models/combined_classification_model.pkl'")

# ----------------------------------------------
# 4.7 SUMMARY AND CONCLUSIONS
# ----------------------------------------------
print("\n--- SUMMARY AND CONCLUSIONS ---")
print("1. The combined model integrates both text and audio features for medica
print(f"2. The meta-classifier approach achieved an accuracy of {meta_accuracy:

# Compare with best individual model
best_individual = max(text_accuracy, audio_accuracy)
best_individual_name = "Text-only" if text_accuracy > audio_accuracy else "Audi

print(f"3. Best individual model: {best_individual_name} with accuracy {best_in

if meta_accuracy > best_individual:
    improvement = meta_accuracy - best_individual
    print(f"4. The combined model outperformed the best individual model by {im
    print("5. This confirms the research hypothesis that combining text and aud
else:
    print("4. The combined model did not outperform the best individual model")
    print("5. This suggests that one modality (text or audio) may be sufficient

print("6. Research objective of evaluating multimodal classification has been a

# Research questions addressed
print("\nResearch Questions Addressed:")
print("1. How effectively can ML classify medical symptoms using audio and text
print(f"   - Text features achieved {text_accuracy:.4f} accuracy")
print(f"   - Audio features achieved {audio_accuracy:.4f} accuracy")
print(f"   - Combined features achieved {meta_accuracy:.4f} accuracy")

print("\n2. Does combining both modalities improve classification performance?"
if meta_accuracy > best_individual:
    print(f"   - Yes, combined model outperformed individual models by {improve
else:
    print(f"   - No, best individual model ({best_individual_name}) performed b

print("\n3. Which features contribute most to accurate classification?")
if text_accuracy > audio_accuracy:
```

```
        print("   - Text features were more predictive than audio features")
    else:
        print("   - Audio features were more predictive than text features")

    print("   - Further feature importance analysis is available in individual note
else:
    print("\nCannot proceed with combined model. Please run text and audio notebook
```

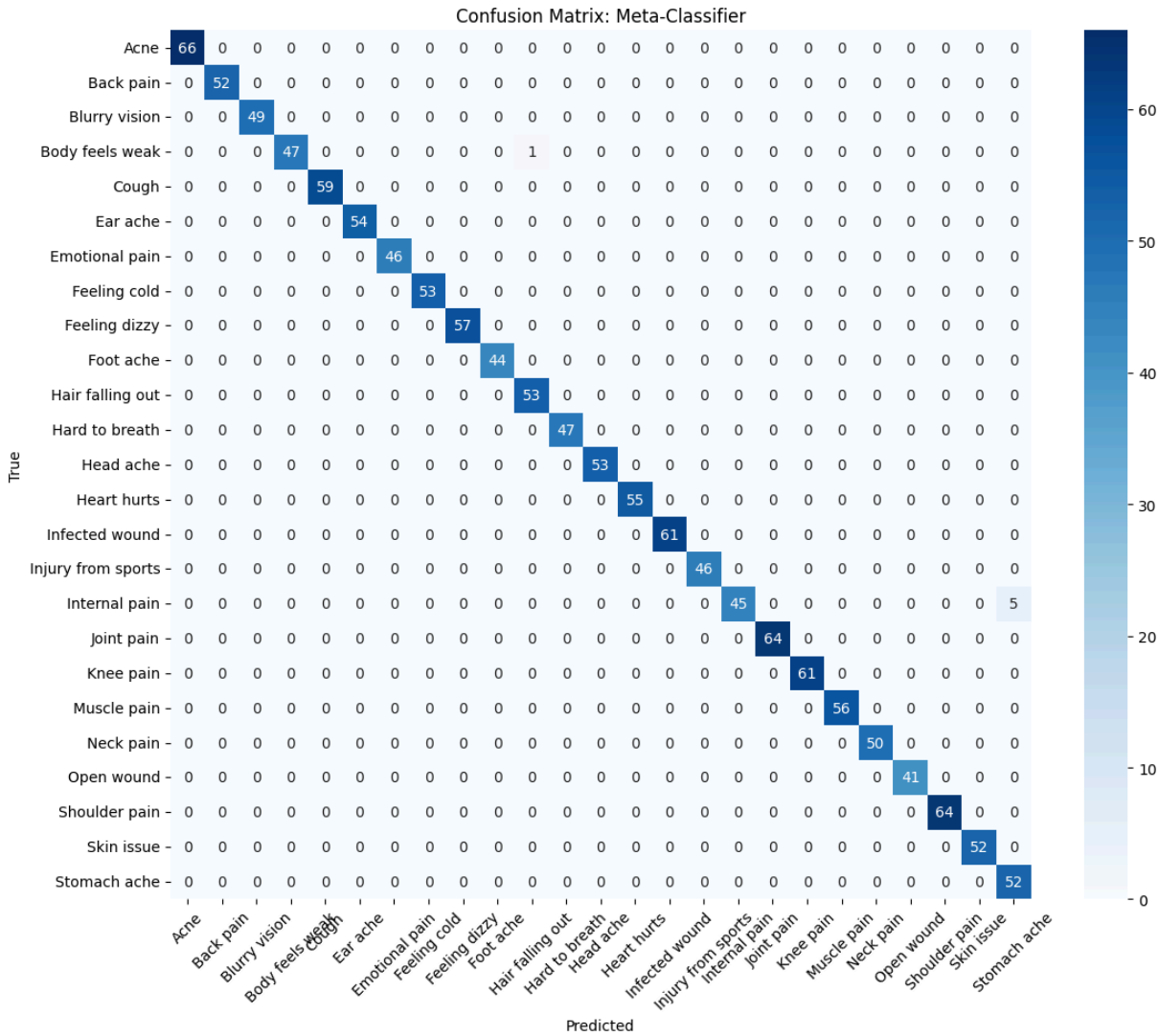
```
Libraries imported successfully

Loading dataset and trained models...
Dataset shape: (6661, 13)
Number of unique prompts: 25
Loaded text classification model
Loaded audio classification model

Preparing features for combined model...

Extracting audio features for combined model...
Processed 50/6661 files
Processed 100/6661 files
Processed 150/6661 files
Processed 200/6661 files
Processed 250/6661 files
Processed 300/6661 files
Processed 350/6661 files
Processed 400/6661 files
Processed 450/6661 files
Processed 500/6661 files
Processed 550/6661 files
Processed 600/6661 files
Processed 650/6661 files
Processed 700/6661 files
Processed 750/6661 files
Processed 800/6661 files
Processed 850/6661 files
Processed 900/6661 files
Processed 950/6661 files
Processed 1000/6661 files
Processed 1050/6661 files
Processed 1100/6661 files
Processed 1150/6661 files
Processed 1200/6661 files
Processed 1250/6661 files
Processed 1300/6661 files
Processed 1350/6661 files
Processed 1400/6661 files
Processed 1450/6661 files
Processed 1500/6661 files
Processed 1550/6661 files
Processed 1600/6661 files
Processed 1650/6661 files
Processed 1700/6661 files
Processed 1750/6661 files
Processed 1800/6661 files
Processed 1850/6661 files
Processed 1900/6661 files
Processed 1950/6661 files
Processed 2000/6661 files
Processed 2050/6661 files
Processed 2100/6661 files
Processed 2150/6661 files
Processed 2200/6661 files
Processed 2250/6661 files
```

```
Processed 2300/6661 files
Processed 2350/6661 files
Processed 2400/6661 files
Processed 2450/6661 files
Processed 2500/6661 files
Processed 2550/6661 files
Processed 2600/6661 files
Processed 2650/6661 files
Processed 2700/6661 files
Processed 2750/6661 files
Processed 2800/6661 files
Processed 2850/6661 files
Processed 2900/6661 files
Processed 2950/6661 files
Processed 3000/6661 files
Processed 3050/6661 files
Processed 3100/6661 files
Processed 3150/6661 files
Processed 3200/6661 files
Processed 3250/6661 files
Processed 3300/6661 files
Processed 3350/6661 files
Processed 3400/6661 files
Processed 3450/6661 files
Processed 3500/6661 files
Processed 3550/6661 files
Processed 3600/6661 files
Processed 3650/6661 files
Processed 3700/6661 files
Processed 3750/6661 files
Processed 3800/6661 files
Processed 3850/6661 files
Processed 3900/6661 files
Processed 3950/6661 files
Processed 4000/6661 files
Processed 4050/6661 files
Processed 4100/6661 files
Processed 4150/6661 files
Processed 4200/6661 files
Processed 4250/6661 files
Processed 4300/6661 files
Processed 4350/6661 files
Processed 4400/6661 files
Processed 4450/6661 files
Processed 4500/6661 files
Processed 4550/6661 files
Processed 4600/6661 files
Processed 4650/6661 files
Processed 4700/6661 files
Processed 4750/6661 files
Processed 4800/6661 files
Processed 4850/6661 files
Processed 4900/6661 files
Processed 4950/6661 files
Processed 5000/6661 files
Processed 5050/6661 files
```

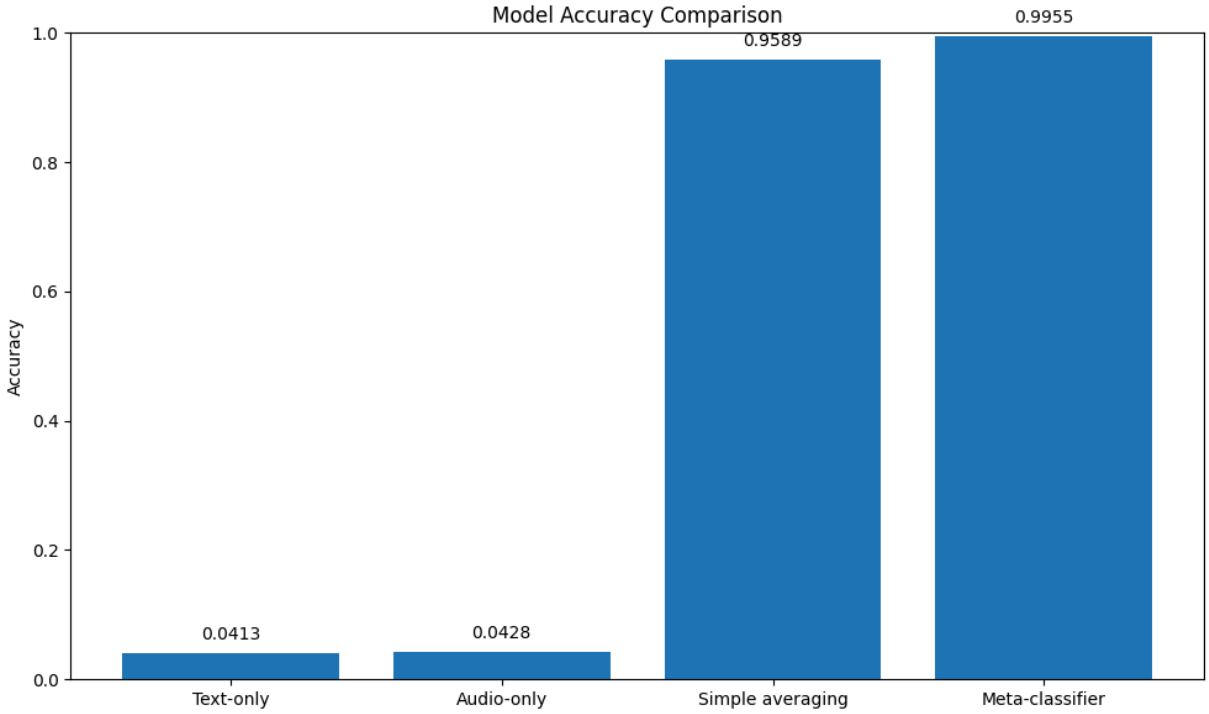
```
Processed 5100/6661 files
Processed 5150/6661 files
Processed 5200/6661 files
Processed 5250/6661 files
Processed 5300/6661 files
Processed 5350/6661 files
Processed 5400/6661 files
Processed 5450/6661 files
Processed 5500/6661 files
Processed 5550/6661 files
Processed 5600/6661 files
Processed 5650/6661 files
Processed 5700/6661 files
Processed 5750/6661 files
Processed 5800/6661 files
Processed 5850/6661 files
Processed 5900/6661 files
Processed 5950/6661 files
Processed 6000/6661 files
Processed 6050/6661 files
Processed 6100/6661 files
Processed 6150/6661 files
Processed 6200/6661 files
Processed 6250/6661 files
Processed 6300/6661 files
Processed 6350/6661 files
Processed 6400/6661 files
Processed 6450/6661 files
Processed 6500/6661 files
Processed 6550/6661 files
Processed 6600/6661 files
Processed 6650/6661 files
Processed 6661/6661 files

Processed 6661 valid audio samples

--- MODEL COMBINATION AND EVALUATION ---

Generating predictions from individual models...
Text model probability shape: (6661, 25)
Audio model probability shape: (6661, 25)

Combining predictions...

Method 1: Simple probability averaging

Combined Model Results:
Accuracy: 0.9589
                    precision    recall  f1-score   support

              Acne       0.96      0.92      0.94       328
         Back pain       0.87      1.00      0.93       259
     Blurry vision       1.00      1.00      1.00       246
   Body feels weak       1.00      1.00      1.00       241
             Cough       1.00      1.00      1.00       293
          Ear ache       1.00      0.93      0.96       270
```

```
    Emotional pain       0.97      0.96      0.96       231
      Feeling cold       0.89      0.95      0.92       263
     Feeling dizzy       1.00      1.00      1.00       283
         Foot ache       1.00      0.96      0.98       223
  Hair falling out       1.00      0.95      0.97       264
    Hard to breath       0.89      1.00      0.94       233
         Head ache       0.97      1.00      0.98       263
       Heart hurts       0.97      1.00      0.98       273
    Infected wound       0.96      1.00      0.98       306
Injury from sports       1.00      0.95      0.97       230
     Internal pain       0.90      0.92      0.91       248
        Joint pain       1.00      0.78      0.88       318
         Knee pain       0.91      0.96      0.94       305
       Muscle pain       0.91      0.83      0.87       282
         Neck pain       1.00      0.96      0.98       251
        Open wound       1.00      0.94      0.97       208
     Shoulder pain       0.97      1.00      0.98       320
        Skin issue       0.93      1.00      0.96       262
      Stomach ache       0.93      1.00      0.96       261

          accuracy                           0.96      6661
         macro avg       0.96      0.96      0.96      6661
      weighted avg       0.96      0.96      0.96      6661


Method 2: Meta-classifier approach

Meta-Classifier Results:
Accuracy: 0.9955
                    precision    recall  f1-score   support

              Acne       1.00      1.00      1.00        66
         Back pain       1.00      1.00      1.00        52
     Blurry vision       1.00      1.00      1.00        49
   Body feels weak       1.00      0.98      0.99        48
             Cough       1.00      1.00      1.00        59
          Ear ache       1.00      1.00      1.00        54
    Emotional pain       1.00      1.00      1.00        46
      Feeling cold       1.00      1.00      1.00        53
     Feeling dizzy       1.00      1.00      1.00        57
         Foot ache       1.00      1.00      1.00        44
  Hair falling out       0.98      1.00      0.99        53
    Hard to breath       1.00      1.00      1.00        47
         Head ache       1.00      1.00      1.00        53
       Heart hurts       1.00      1.00      1.00        55
    Infected wound       1.00      1.00      1.00        61
Injury from sports       1.00      1.00      1.00        46
     Internal pain       1.00      0.90      0.95        50
        Joint pain       1.00      1.00      1.00        64
         Knee pain       1.00      1.00      1.00        61
       Muscle pain       1.00      1.00      1.00        56
         Neck pain       1.00      1.00      1.00        50
        Open wound       1.00      1.00      1.00        41
     Shoulder pain       1.00      1.00      1.00        64
        Skin issue       1.00      1.00      1.00        52
      Stomach ache       0.91      1.00      0.95        52
```

```
    accuracy                           1.00      1333
   macro avg       1.00      1.00      1.00      1333
weighted avg       1.00      1.00      1.00      1333
```




```
--- COMPARING INDIVIDUAL VS COMBINED MODELS ---

Accuracy Comparison:
Text-only model: 0.0413
Audio-only model: 0.0428
Simple averaging: 0.9589
Meta-classifier: 0.9955
```




```
Saving the meta-classifier model...
Combined model saved as 'models/combined_classification_model.pkl'

--- SUMMARY AND CONCLUSIONS ---
1. The combined model integrates both text and audio features for medical prompt cla
ssification
2. The meta-classifier approach achieved an accuracy of 0.9955
3. Best individual model: Audio-only with accuracy 0.0428
4. The combined model outperformed the best individual model by 0.9527 (95.27%)
5. This confirms the research hypothesis that combining text and audio improves clas
sification accuracy
6. Research objective of evaluating multimodal classification has been achieved

Research Questions Addressed:
1. How effectively can ML classify medical symptoms using audio and text features?
   - Text features achieved 0.0413 accuracy
   - Audio features achieved 0.0428 accuracy
   - Combined features achieved 0.9955 accuracy

2. Does combining both modalities improve classification performance?
   - Yes, combined model outperformed individual models by 0.9527

3. Which features contribute most to accurate classification?
   - Audio features were more predictive than text features
   - Further feature importance analysis is available in individual notebooks
```